Regional DP Patterns

Faster transitions, subsequence counting, and getting the answer back

Learning goals

- Spot transitions that sum or maximise over a **contiguous range** of states
- Speed them up with **prefix sums** and **monotonic deques**
- Count subsequences: patterns, distinct subsequences, wildcards
- Reconstruct optimal solutions (parent pointers, re-tracing, lexicographic order)
- Recognise when a correct $O(n^2)$ DP must become $O(n)$ or $O(n \log n)$

The regional DP situation

- You found a correct DP. It is $O(n \cdot k^2)$ or $O(n^2)$ and too slow by a factor n
- In most regional problems the fix is one of:
    i. transition over a **range** → prefix sums
    ii. max/min over a **sliding window** → monotonic deque
    iii. transition over "best value with key < x" → Fenwick / segment tree
    iv. smarter state (drop a dimension)
- Today: 1, 2, plus counting and reconstruction

Step 1: write the slow DP first

```
dp[i][j] = Σ_{x = 0}^{a_i} dp[i−1][j − x]          O(N·K·K)
```

- Write it down *exactly*, including the range of the inner variable
- Look at the inner loop: is it a **contiguous range** of the previous row?
- If yes: precompute prefix sums of the previous row → O(1) per state

Prefix sums: EDPC M *Candies*

N children, child i gets $0..a_i$ candies, K total. Count ways mod p.

```
for (int i = 0; i < n; i++) {
    pre[0] = 0;
    for (int j = 0; j <= k; j++) pre[j+1] = (pre[j] + dp[j]) % MOD;
    for (int j = 0; j <= k; j++) {
        int lo = max(0, j - a[i]);
        dp[j] = (pre[j+1] - pre[lo] + MOD) % MOD;   // Σ dp[lo..j]
    }
}
```

- $O(N \cdot K)$ instead of $O(N \cdot K^2)$; note `+ MOD` before `%`

Monotonic deque refresher

- Sliding window maximum in O(1) amortised per step
- Deque holds indices with **decreasing** values
- Push: pop smaller values from the back; Pop: drop indices outside the window

```
window → [ 7  3  5  2 ]   deque (values): 7 5 2
add 6  →   pop 2, pop 5 → 7 6
```

Sliding-window DP: CF 372C *Fireworks*

- Positions 1..n; between fireworks you move at most `w = d·Δt`
- `dp_i[x] = b_i − |a_i − x| + max_{|y−x| ≤ w} dp_{i−1}[y]`
- Inner max is over a **window of width 2w+1** sliding with x → deque
- O(m·n) instead of O(m·n·w)
- Cap `w` at n: `(t − prevT)·d` can be 10^{14}

Fireworks: the window loop

```
deque<int> dq; int right = 1;
for (int x = 1; x <= n; x++) {
    while (right <= n && right <= x + w) {          // extend
        while (!dq.empty() && dp[dq.back()] <= dp[right]) dq.pop_back();
        dq.push_back(right++);
    }
    while (dq.front() < x - w) dq.pop_front();      // shrink
    nd[x] = dp[dq.front()] + b - llabs(a - x);
}
swap(dp, nd);
```

Other window shapes

- "Pick at least one in every window of length k" (CF 1077F2 *Pictures with Kittens*):
  `dp[j][i] = a_i + max_{i−k ≤ p < i} dp[j−1][p]` → deque per layer
- "Last jump at most k back": `dp[i] = c_i + min_{i−k ≤ p < i} dp[p]`
- Non-contiguous but "key < x" → Fenwick/segment tree over keys (Lecture 6)

DP on subsequences: the counting family

Question	State
How many times does `t` occur as a subsequence of `s`?	`dp[j]` = ways to match `t[0..j)`
How many **distinct** subsequences?	`ways`, plus `last[c]`
Total `abc` subsequences over all `?` fillings	counts of `a`, `ab`, `abc`, strings
Longest common / increasing subsequence	classic + reconstruction

Occurrences of t as a subsequence

```
vector<ll> dp(m + 1, 0); dp[0] = 1;        // dp[j]: ways to match t[0..j)
for (char c : s)
    for (int j = m; j >= 1; j--)            // backwards: use c at most once
        if (t[j-1] == c) dp[j] = (dp[j] + dp[j-1]) % MOD;
// answer dp[m]
```

- Same trick as 0/1 knapsack: iterate j downwards
- $O(|s|\cdot|t|)$; with a fixed short t (like `"abc"`) it is $O(|s|)$

Distinct subsequences

- `ways` = distinct subsequences of the prefix (incl. empty)
- Appending c: every old subsequence, with or without c → `2·ways`
- Over-counted: those ending in c already created at c's previous occurrence

```
ll nw = (2 * ways - last[c] + MOD) % MOD;
last[c] = ways;            // ways *before* this occurrence
ways = nw;
// answer: ways − 1 (drop empty)
```

Wildcards: CF 1426F

- Count "abc" subsequences summed over all 3^q fillings of `?`
- Track **totals over all strings**: `cnt` (strings), `a`, `ab`, `abc`

```
'a':  a   += cnt
'b':  ab  += a
'c':  abc += ab
'?':  three copies at once:
      a' = 3a + cnt,  ab' = 3ab + a,  abc' = 3abc + ab,  cnt' = 3cnt
```

- Order matters: compute all new values from **old** ones

Reconstruction: the three methods

1. **Parent pointers**: store the argmax for each state, follow back
2. **Re-trace the table**: from the final state, find any predecessor whose value + transition equals the current value
3. **Forward greedy over a suffix DP**: compute dp from the end, then build the answer front to back → easy **lexicographically smallest** solution

LIS in O(n log n) with reconstruction

```
for (int i = 0; i < n; i++) {
    int k = lower_bound(tailVal.begin(), tailVal.end(), a[i])
            - tailVal.begin();              // strict; upper_bound: non-decr.
    if (k == (int)tailVal.size()) { tailVal.push_back(a[i]); tailIdx.push_back(i); }
    else { tailVal[k] = a[i]; tailIdx[k] = i; }
    parent[i] = k > 0 ? tailIdx[k-1] : -1;
}
// follow parent[] from tailIdx.back(), then reverse
```

- `tailVal` itself is **not** an LIS — only its length is meaningful

LCS reconstruction by re-tracing

```
string res; int i = n, j = m;
while (i > 0 && j > 0) {
    if (s[i-1] == t[j-1]) { res += s[i-1]; i--; j--; }
    else if (dp[i-1][j] >= dp[i][j-1]) i--;
    else j--;
}
reverse(res.begin(), res.end());
```

- No extra memory: the table already encodes the decisions
- EDPC F *LCS*

Lexicographically smallest answer

- Compute `best[i]` = optimal value of the **suffix** starting at state i
- Walk forward: at each step choose the smallest symbol whose next state keeps `best` optimal
- Works because the decision is made with full knowledge of the future
- Common in regional "output the solution; if many, the lexicographically smallest"

Complexity summary

Pattern	Before	After
Range-sum transition	$O(n \cdot k^2)$	$O(n \cdot k)$
Sliding-window max	$O(n \cdot w)$ per layer	$O(n)$ per layer
Subsequence occurrences	—	$O($
Distinct subsequences	—	$O(n)$
LIS	$O(n^2)$	$O(n \log n)$

Common pitfalls

- Negative values after modular subtraction: add `MOD` before `%`
- Deque window bounds off by one (`< x - w` vs `<= x - w`)
- Window width overflow (`Δt · d` can be 10^{14}) — cap at n
- Updating `last[c]` before computing the new value
- `lower_bound` vs `upper_bound` for strict vs non-strict LIS
- Reconstruction from the *optimised* DP needs the stored argmax

Practice problems

- AtCoder EDPC M — Candies; EDPC F — LCS
- Codeforces 372C — Watching Fireworks is Fun
- Codeforces 1077F2 — Pictures with Kittens (hard version)
- Codeforces 1426F — Number of Subsequences
- Codeforces 1183H — Subsequences (hard version)
- CSES 1145 — Increasing Subsequence
- CSES 1093 — Two Sets II
- CSES 1644 — Maximum Subarray Sum II (prefix sums + window min)

Summary

- Write the slow DP precisely, then look at the shape of the inner loop
- Contiguous range sum → prefix sums; window extremum → monotonic deque
- Subsequence counting: process characters left to right, maintain counts per pattern prefix
- Reconstruction: parents, re-tracing, or suffix DP for lexicographic order
- Next: coordinate compression, sweep line, and segment trees in practice